Django

Django is a high-level Python web framework that helps you build web applications quickly and securely.

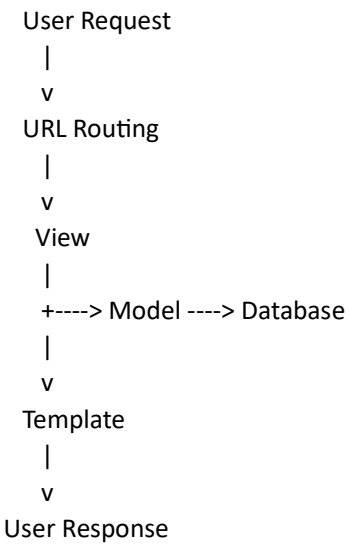
Django Architecture (MVT)

Django follows the MVT (Model–View–Template) architecture pattern.

The purpose of MVT is to separate:

- Data management (Model)
- Business logic (View)
- User interface (Template)

This separation makes applications easier to develop, test, and maintain.



1. Model

Also known as data layer . it manages data and interacts with the database. Django models are written as Python classes.

- Define database tables
- Store data
- Retrieve data
- Update data
- Delete data

2. View

Also Known as logic layer. A View contains the business logic. It receives requests and decides what response to send back.

- Receive request
- Execute logic
- Fetch data from Model
- Send data to Template

- Return response

3. Template

A Template handles the presentation layer. Templates are usually HTML files. They display data received from Views. it display data to users

- Display Data to Users
- Create the User Interface
- Render Dynamic Content

Project

A Project is the entire Django website/application. One project can have many apps .It contains:

- Global settings
- Main URL configuration
- Database configuration
- Multiple apps

App

An App is a module that performs a specific function within the project. App belongs to a project

Core Files Explanation:

1.settings.py - This is the main configuration file of the project. is the heart of the Django project.(Configuration File)

Responsibilities:

- Database configuration
- Installed apps
- Middleware
- Templates configuration
- Static files settings
- Security settings
- Debug mode

2.urls.py - This file manages URL routing.(Routing File)

Responsibilities:

- Maps URLs to Views
- Directs requests to the correct function or class
- Organize application routes

3. views.py - This file contains the application's business logic. is created inside an app (Logic File)

Responsibilities:

- Receive requests
- Process data
- Communicate with Models
- Return responses

4. manage.py - Used to run Django commands.(Command Utility)

Responsibilities:

- Run the development server
- Create apps

- Apply database migrations
- Create superusers
- Execute Django commands